
Page Replacement

Chap 21, 22, 23

Page Replacement
for Virtual Memory

Demand Paging

these can load

- **Paging (Demand paging) system**

- Partition the program into the same size blocks (pages)
- Loading of executable program
 - Initially, load pages only as they are needed
 - During execution, load the pages when they are demanded (referenced)
 - Pages that are never accessed are never loaded into physical memory
- With each page table entry a **present (residence)** bit is associated

- **Present = true:** in-memory, **memory resident**

- **Present = false:** not-in-memory

- Initially present bit is set to false on all entries
- During MMU address translation, if present bit in page table entry is **false** $\Rightarrow$ page fault

access $\rightarrow$ *page fault*

all page not loaded into physical memory

trap

Steps in Handling a Page Fault

4KB page

```

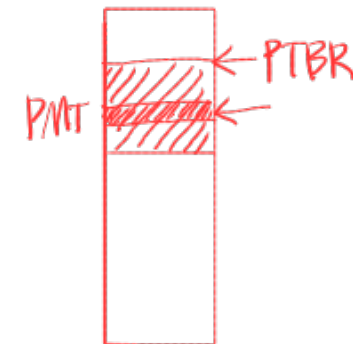
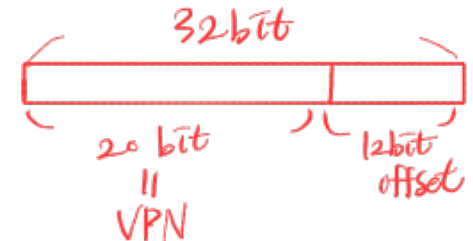
1  VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2  (Success, TlbEntry) = TLB_Lookup(VPN)
3  if (Success == True) // TLB Hit
4      if (CanAccess(TlbEntry.ProtectBits) == True)
5          Offset = VirtualAddress & OFFSET_MASK
6          PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7          Register = AccessMemory(PhysAddr)
8      else
9          RaiseException(PROTECTION_FAULT)
10 else // TLB Miss
11     PTEAddr = PTBR + (VPN * sizeof(PTE))
12     PTE = AccessMemory(PTEAddr)
13     if (PTE.Valid == False)
14         RaiseException(SEGMENTATION_FAULT)
15     else
16         if (CanAccess(PTE.ProtectBits) == False)
17             RaiseException(PROTECTION_FAULT)
18         else if (PTE.Present == True)
19             // assuming hardware-managed TLB
20             TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21             RetryInstruction()
22         else if (PTE.Present == False)
23             RaiseException(PAGE_FAULT)

```

Handwritten notes:

- 20 1111 ... 0000* (above line 1)
- 12bit* (next to line 1)
- TLB miss* (next to line 10)
- physical frame number* (next to line 6)
- invalid page* (next to line 13)
- Trap* (next to line 23)

Page-Fault Control Flow Algorithm (Hardware)



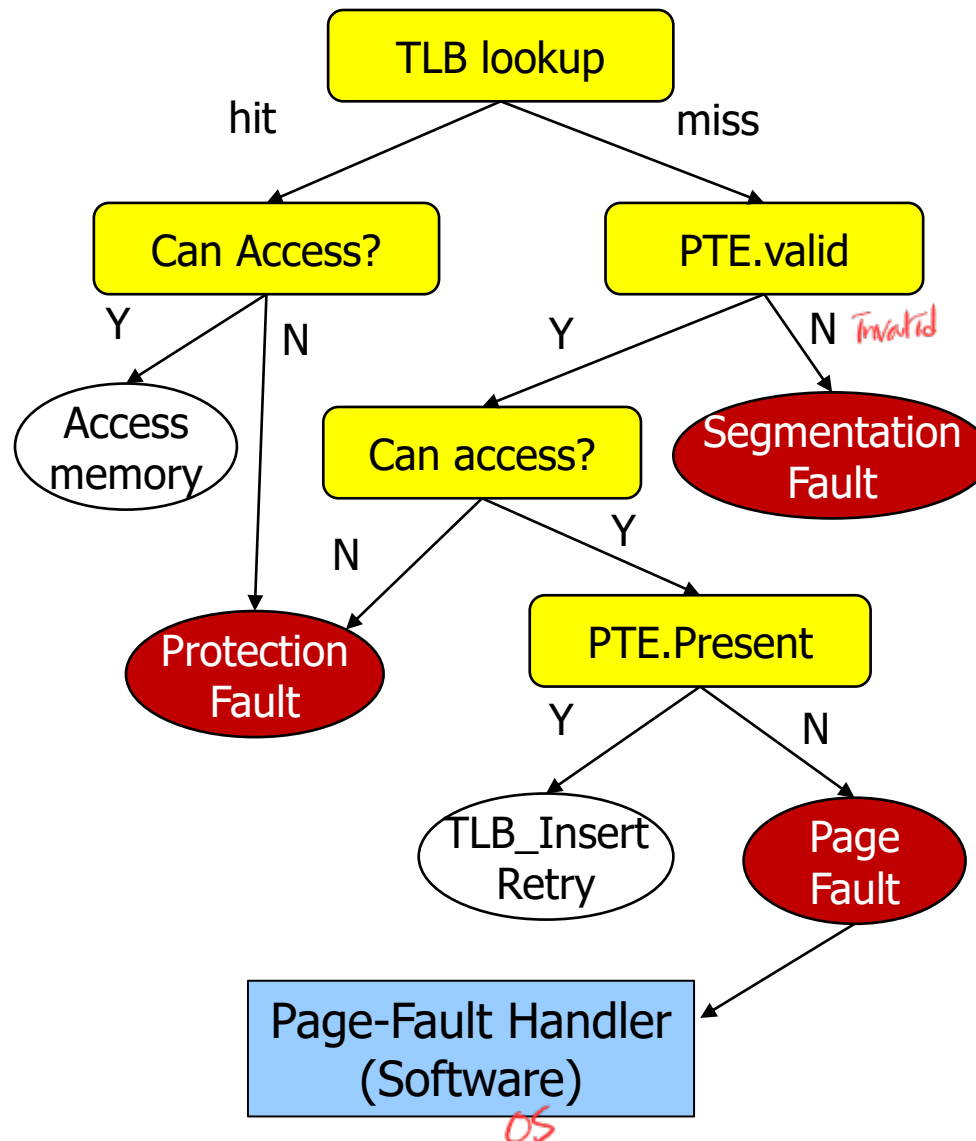
Page-Fault Handler (Software)

```

1  PFN = FindFreePhysicalPage()
2  if (PFN == -1) // no free page found
3      PFN = EvictPage() // run replacement algorithm LRU
4  DiskRead(PTE.DiskAddr, pfn) // sleep (waiting for I/O)
5  PTE.present = True // update page table with present
6  PTE.PFN = PFN // bit and translation (PFN)
7  RetryInstruction() // retry instruction

```

Steps in Handling a Page Fault



Performance of Demand Paging

- **Average memory access time (AMAT)**
 - $AMAT = T_M + (P_{MISS} \times T_D)$
 - Memory access time (T_M): Assume 100 ns
 - Average paging service time (T_D): Assume 10 ms
 - Page fault rate (P_{MISS}): $0 \leq P_{MISS} \leq 1$
 - $AMAT = 100\text{ns} + P_{MISS} \times 10\text{ms}$
 - When $P_{MISS} = 0.1$, $AMAT = 1.0001 \text{ ms}$ ($10,000 \times T_M$)
 - When $P_{MISS} = 0.001$, $AMAT = 10.1 \text{ us}$ ($100 \times T_M$)

$$100\text{ns} = 0.1\mu\text{s}$$

$$100\text{ns}$$

$$= 0.1\mu\text{s}$$

$$= 0.0001\text{ms}$$

$$10\text{ms}$$

$$100000\mu\text{s}$$

$$10000000\text{ns}$$

$$P=0.1) \quad 0.0001\text{ms} + 0.1 \times 10\text{ms}$$

$$= 1.0001\text{ms}$$

$$P=0.001) \quad 0.0001\text{ms} + 0.001 \times 10\text{ms}$$

$$= 0.0101\text{ms}$$

$$= 10.1\mu\text{s}$$

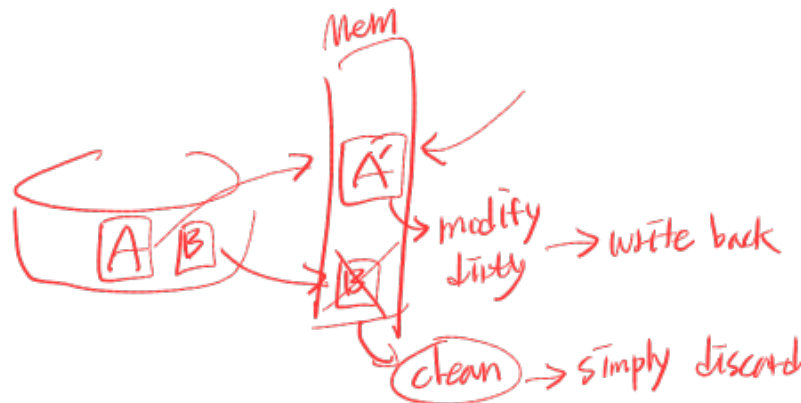
$$1\text{ms} = 1000\mu\text{s}$$

$$\mu\text{s} = 0.001\text{ms}$$

$$1\text{ms} = 1000\mu\text{s}$$

Page Replacement

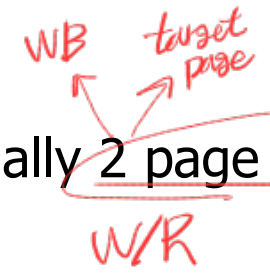
- Prevent over-allocation of memory
- Use **modify** (**update**, **dirty**) bit to reduce overhead of page transfers
 - only modified pages are written to disk
 - If $\text{modify} == 1$, the contents of the page in memory and in disk are not same
 - Write-back (to disk) is necessary for the page
- Large virtual memory can be provided on a smaller physical memory



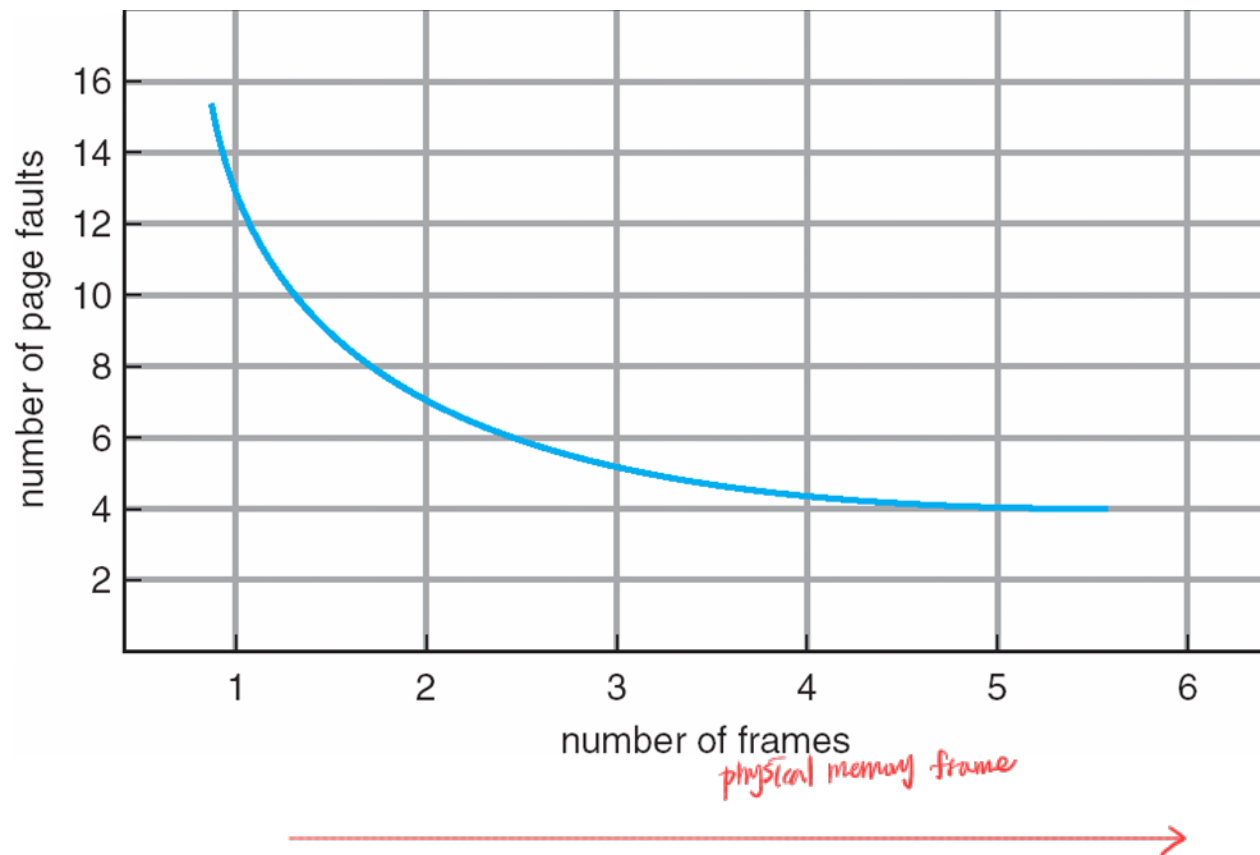
Page Replacement

- 
1. Find the location of the ^{desired page} desired page on disk
2. Find a free frame:
- If there is a free frame, use it
 - If there is no free frame, use a page replacement algorithm to select a victim frame
 - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Note now potentially 2 page transfers for page fault – increasing AMAT



Graph of Page Faults Versus The Number of Frames



Paging out

- Pages from program binary on disk (code, data)

- can be simply discarded when freeing frame

- Cached Pages of files modified in memory

- write back to the file system

- Pages not associated with a file (stack and heap – anonymous memory) and dirty static data of program

- swapping

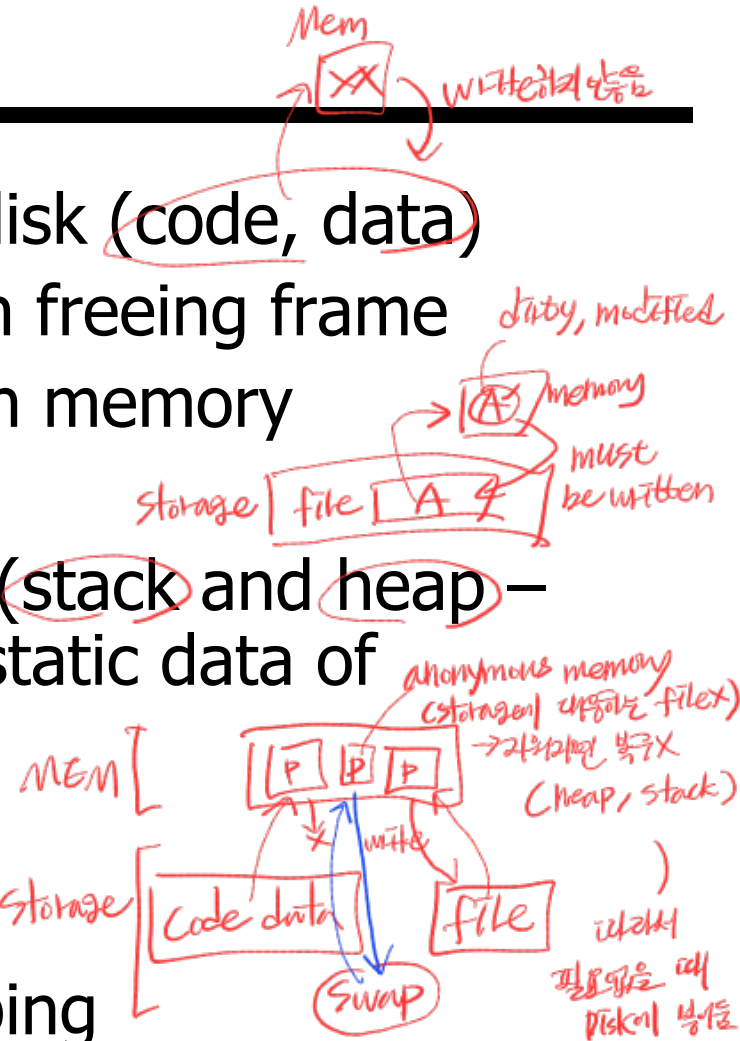
- Mobile systems

- Typically don't support swapping

- Instead, Low Memory Killer

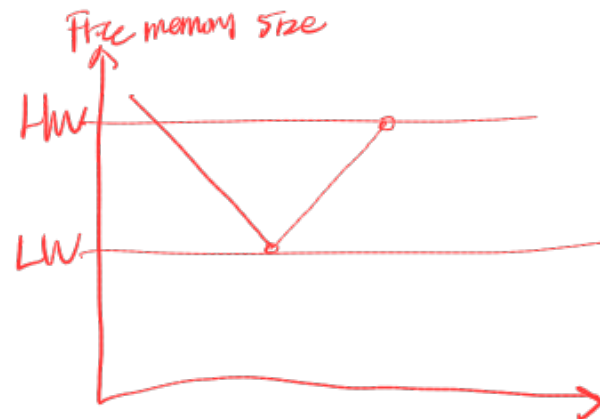
- cf. zram and zswap

compression

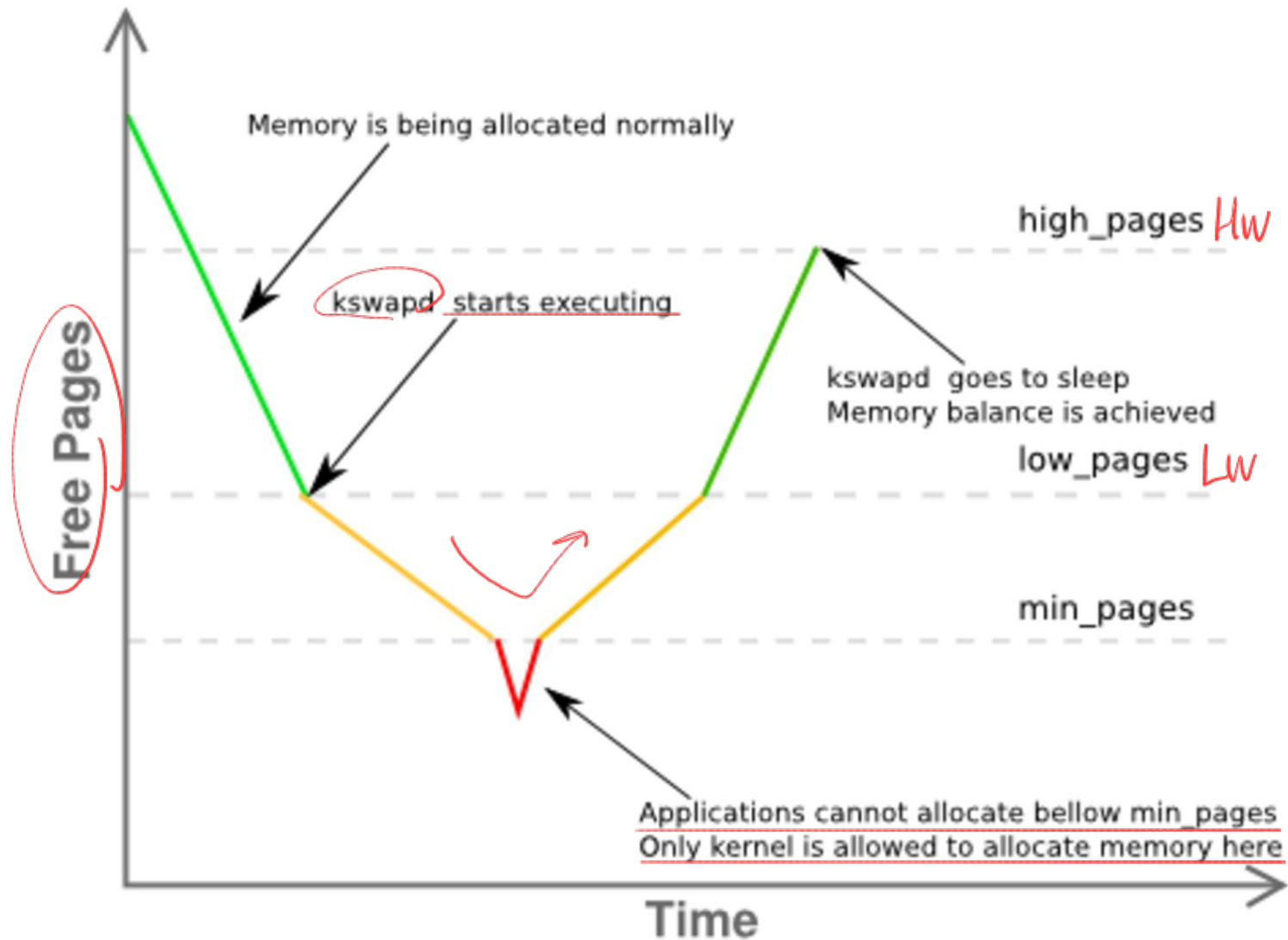


When Replacements Really Occur?

- OS keeps a small portion of memory free more proactively
- **Watermark** scheme
 - high watermark (HW) and low watermark (LW)
 - When OS notices that there are fewer than LW pages available, a background thread (**swap daemon** or **page daemon**) that is responsible for freeing memory runs.
 - The thread evicts pages until there are HW pages available.
 - The background thread then goes to sleep.
 - many systems will **cluster** or **group** a number of pages and write them out at once to the swap partition, thus increasing the efficiency of the disk



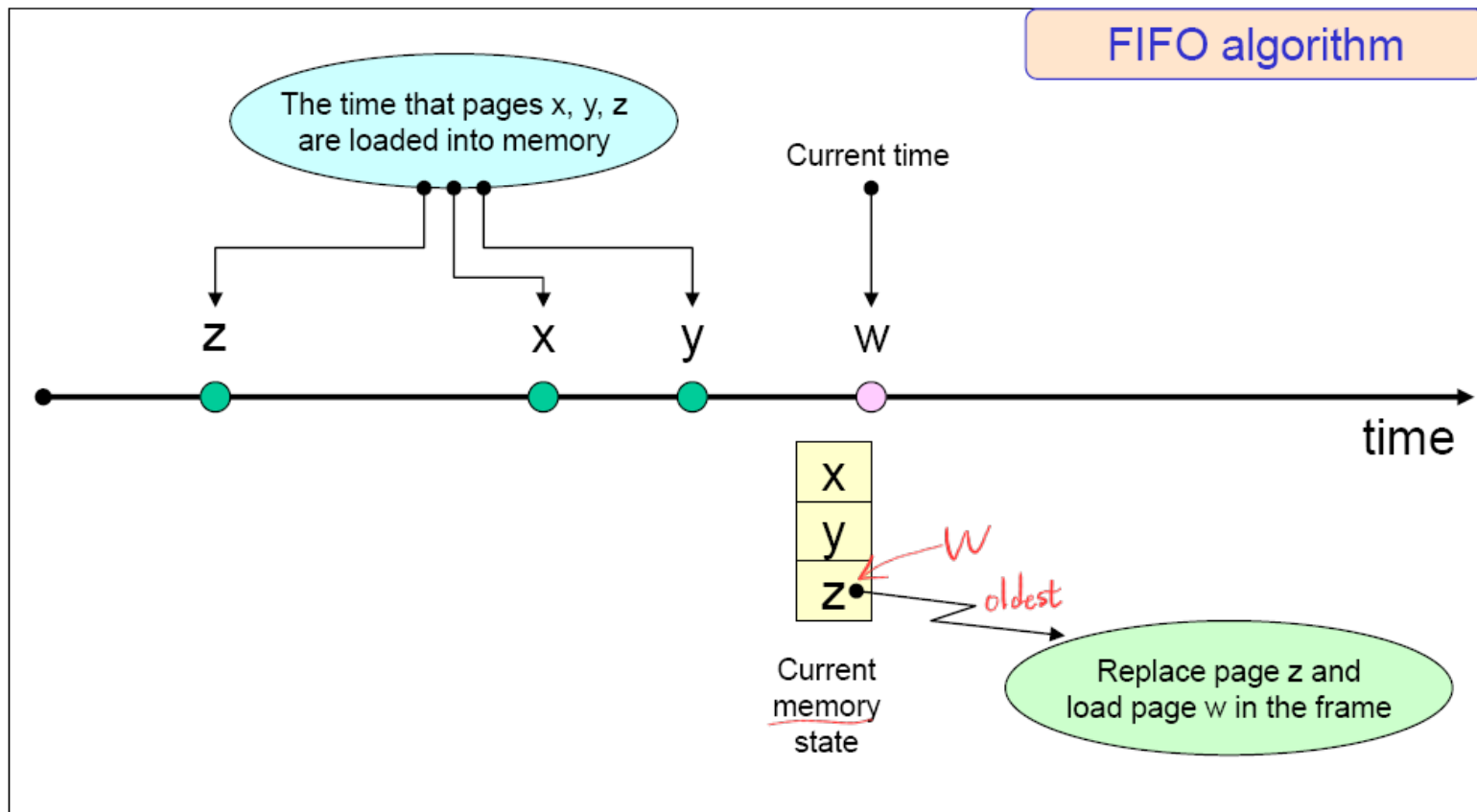
Memory Reclamation at Linux



First-In-First-Out (FIFO) Algorithm

- Choose the page to be replaced based on when the page is previously loaded into memory
- Scheme
 - Replace the oldest page
- Requirements
 - Timestamping (memory load time for each page) is necessary
- Characteristics
 - May replace frequently used pages
- **FIFO anomaly (Belady's anomaly)**
 - In FIFO algorithm, page fault frequency may increase even if more memory frames are allocated

First-In-First-Out (FIFO) Algorithm



First-In-First-Out (FIFO) Algorithm

- FIFO algorithm: Example**

- 4 page frames allocated, initially empty

Access Sequence

$\omega = 1\ 2\ 6\ 1\ 4\ 5\ 1\ 2\ 1\ 4\ 5\ 6\ 4\ 5$

Memory state change (FIFO)

Time	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Ref. string	1	2	6	1	4	5	1	2	1	4	5	6	4	5
Memory state	1	1	1	1	1	5	5	5	5	5	5	5	4	4
		2	2	2	2	2	1	1	1	1	1	1	1	5
			6	6	6	6	6	2	2	2	2	2	2	2
					4	4	4	4	4	4	4	4	6	6
Page fault	F	F	F	<i>Hit</i>	F	F	F	F	<i>Hit</i>	<i>Hit</i>	<i>Hit</i>	F	F	F

▪ Number of page faults: 10

no free frame

First-In-First-Out (FIFO) Algorithm

- FIFO algorithm: Anomaly example

$\omega = 1\ 2\ 3\ 4\ 1\ 2\ 5\ 1\ 2\ 3\ 4\ 5$

Number of page frames: 3

Time	1	2	3	4	5	6	7	8	9	10	11	12
Ref. string	1	2	3	4	1	2	5	1	2	3	4	5
Memory state	1	1	1	4	4	4	5	5	5	5	5	5
		2	2	2	1	1	1	①	1	3	3	3
			3	3	3	2	2	2	②	2	4	4
Page fault	F	F	F	F	F	F	F			F	F	

▪ Number of page faults: 9

Number of page frames: 4

Time	1	2	3	4	5	6	7	8	9	10	11	12
Ref. string	1	2	3	4	1	2	5	1	2	3	4	5
Memory state	1	1	1	1	①	1	5	5	5	5	4	4
		2	2	2	2	②	2	1	1	1	1	5
			3	3	3	3	3	3	2	2	2	2
			4	4	4	4	4	4	4	3	3	3
Page fault	F	F	F	F			F	F	F	F	F	F

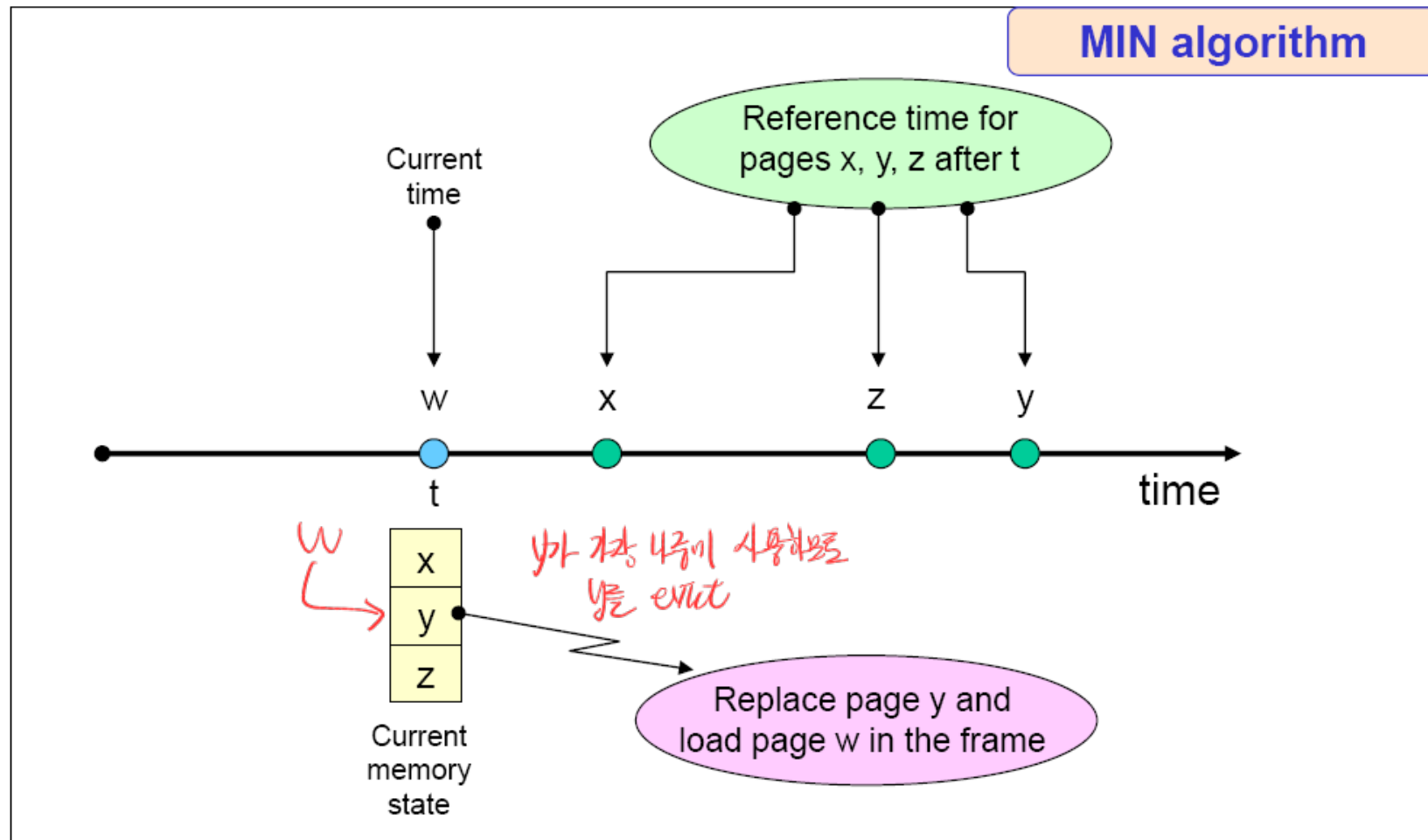
▪ Number of page faults: 10

page frames 늘렸는데
fault가 오히려 늘어남

MIN algorithm (OPT algorithm)

- Proposed by Belady in 1966 ^{optimal}
- Minimizes page fault frequency (proved)
- Scheme
 - Replace the page that will not be used for the longest period of time _{↪ future}
 - Tie-breaking rule
 - Page with greatest (or smallest) page number
- Unrealizable
 - Can be used only when the process's reference string is known a priori _{reference string을 사전에 알고 있어야 가능}
- Usage
 - Performance measurement tool for replacement schemes

MIN algorithm (OPT algorithm)



MIN algorithm (OPT algorithm)

- MIN algorithm: Example**

Unrealistic

- 4 page frames allocated, initially empty

$\omega = 1\ 2\ 6\ 1\ 4\ 5\ 1\ 2\ 1\ 4\ 5\ 6\ 4\ 5$

Memory state change (MIN)

Time	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Ref. string	1	2	6	1	4	5	1	2	1	4	5	6	4	5
Memory state	1	1	1	1	1	1	1	1	1	1	1	6	6	6
		2	2	2	2	2	2	2	2	2	2	2	2	2
			6	6	6	5	5	5	5	5	5	5	5	5
					4	4	4	4	4	4	4	4	4	4
Page fault	F	F	F		F	F						F		

*들끓음 마무리
선택*

▪ Number of page faults: 6

Least Recently Used (LRU) Algorithm

- Choose the page to be replaced based on the reference time
- Scheme
 - Replace the page that has not been used for the longest period of time
- Requirements
 - Timestamping (page reference time) is necessary
- Characteristics
 - Based on program locality
 - Approximates to the performance of MIN algorithm
- Used in most practical systems
- Drawbacks
 - Timestamping overhead at every page reference
 - Number of page faults increases steeply when the process executes large loop with insufficiently allocated memory

→ past history

past history

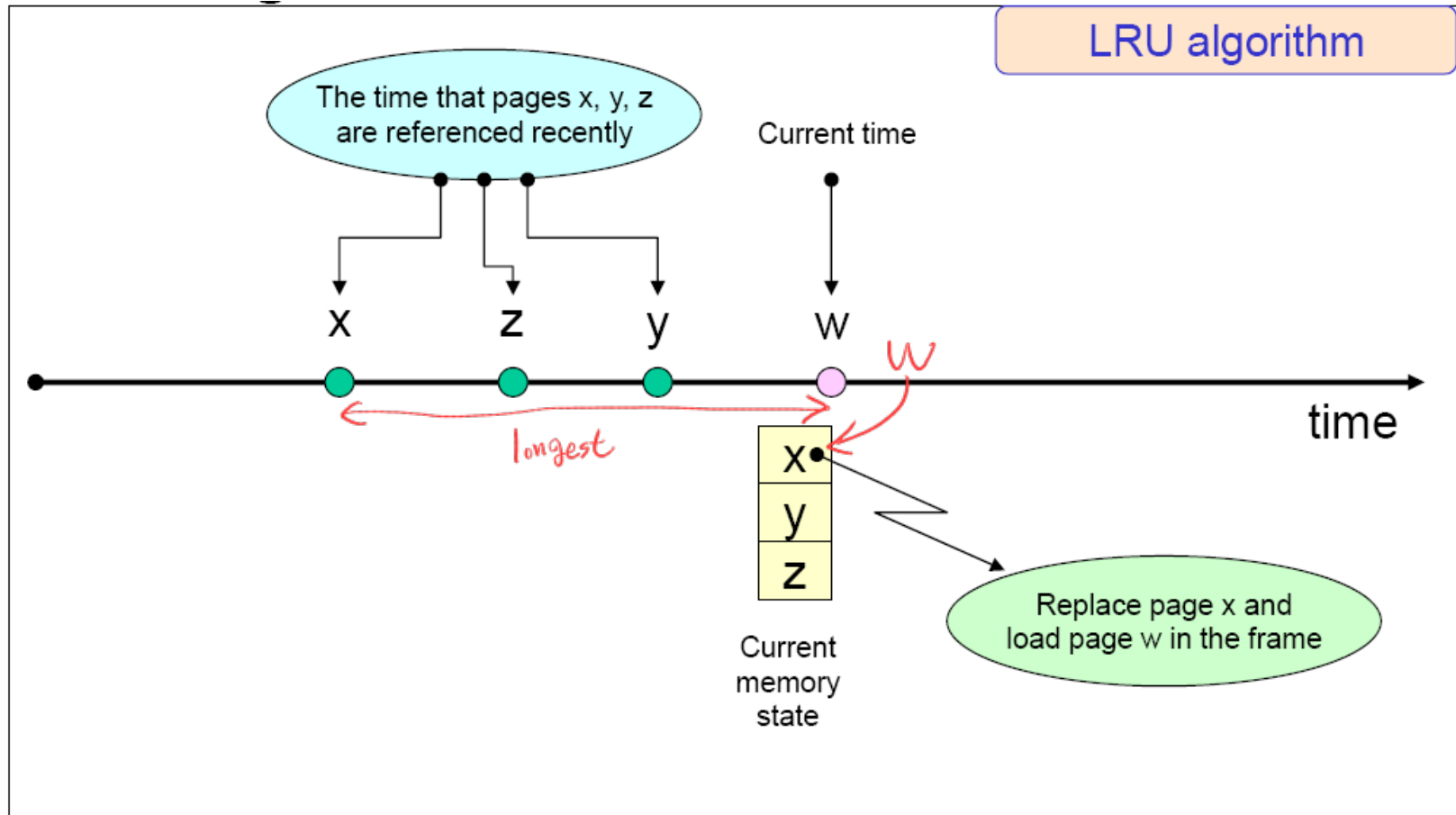
→ will not be used for the long time
→ predict future

similar performance MIN

FIFO : 들어온 순서만 기억

loop를 돌며 reference하는 경우 fault가 늘어날 수 있음

Least Recently Used (LRU) Algorithm



Least Recently Used (LRU) Algorithm

- LRU algorithm: Example**

- 4 page frames allocated, initially empty

$\omega = 1\ 2\ 6\ 1\ 4\ 5\ 1\ 2\ 1\ 4\ 5\ 6\ 4\ 5$

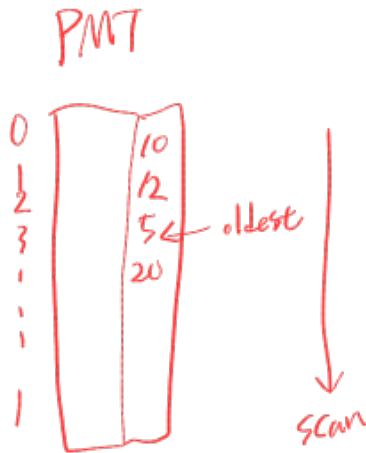
Memory state change (LRU)

Time	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Ref. string	1	2	6	1	4	5	1	2	1	4	5	6	4	5
Memory state	1	1	1	1	1	1	1	1	1	1	1	1	1	1
		2	2	2	2	5	5	5	5	5	5	5	5	5
			6	6	6	6	6	2	2	2	2	6	6	6
					4	4	4	4	4	4	4	4	4	4
Page fault	F	F	F		F	F		F				F		

▪ Number of page faults: 7

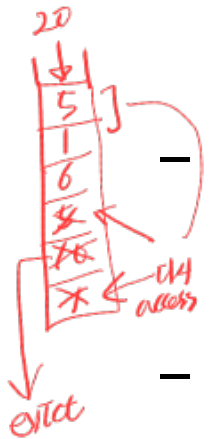
Implementation of LRU algorithm

- By counter
 - Use PMT with count field
 - Increment processor clock or counter for each memory access
 - Record the value of processor clock or counter in the corresponding PMT entry for each page reference
 - Can get the relative order of recent access to each page
 - PMT search for selecting a page to be replaced



Implementation of LRU algorithm

- By stack
 - Stack
 - Stack for each process, whose entry is page number
 - Maintains the stack elements (page numbers) in the order of recent access
 - Can delete an element in the middle of the stack
 - When no page fault
 - Deletes the referenced page number from the stack, and inserts it on top of the stack
 - When page fault
 - Displaces the page whose number is at the bottom of the stack, deletes it from the stack, and inserts incoming page number on top of the stack



LRU Approximation Algorithms

LRU $\begin{cases} \text{count} \\ \text{stack} \end{cases}$ High overhead

- LRU needs special hardware and still slow

- NUR(Not Used Recently) algorithm

- LRU approximation scheme
- Lower overhead than LRU algorithm
- Uses bit vectors

LRU(least) ① 가장 오래 접근하지 않은 것
NUR(Not) ② 최근에 접근하지 않은 것들

- Reference bit vector

$[r, m]$

- Update bit vector

$\begin{cases} \text{modify,} \\ \text{update} \end{cases}$

- Scheme

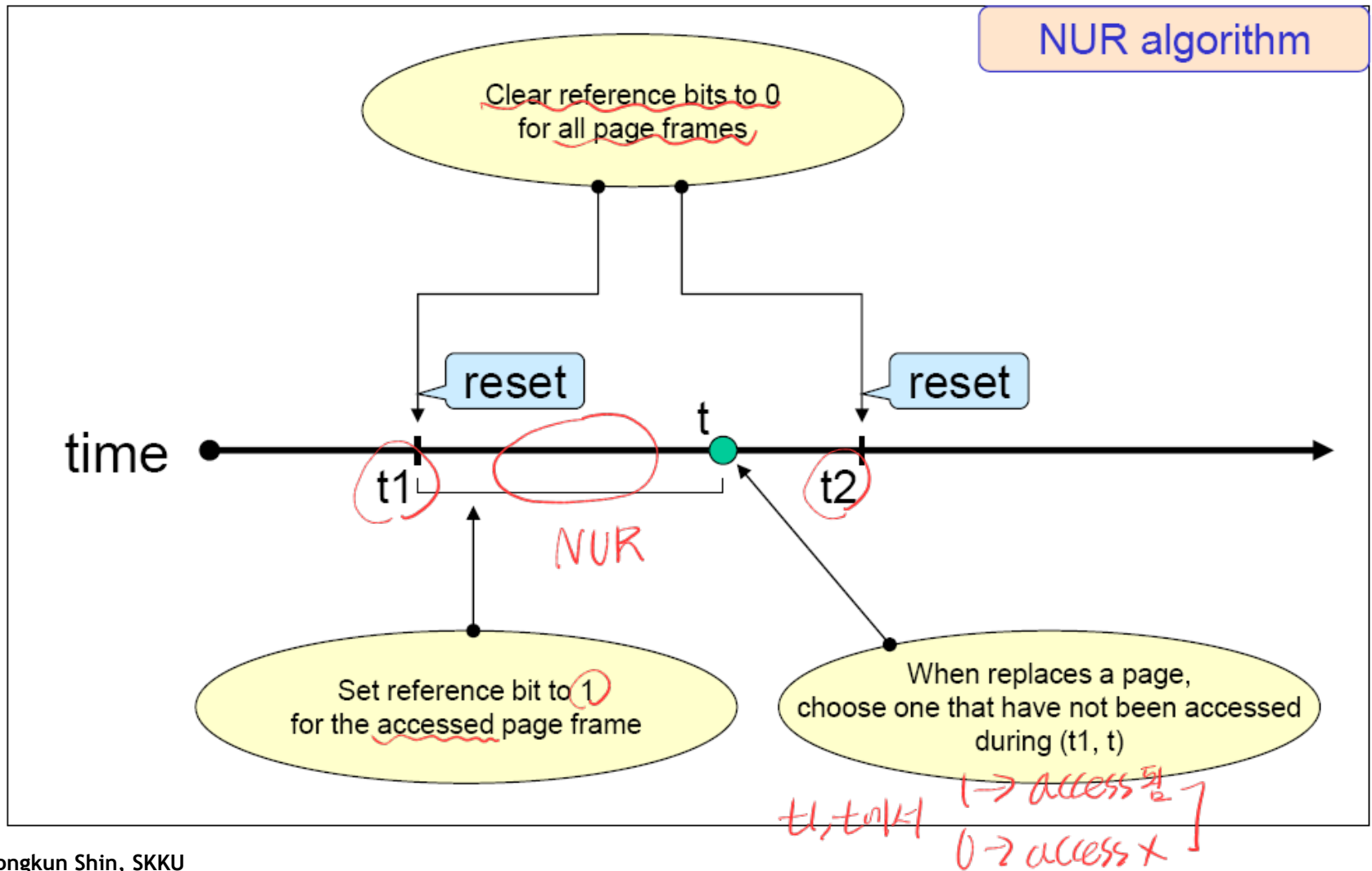
- Check reference/update bit and choose a victim page
- Order of replacement (reference bit: r, update bit: m)

이름 access되지 않은 것라고 예측

- ① Replace the page with $(r, m) = (0, 0)$ → Victim (not accessed, not updated)
- ② Replace the page with $(r, m) = (0, 1)$ → clean (no write cost)
- ③ Replace the page with $(r, m) = (1, 0)$ → dirty (write storage)
- ④ Replace the page with $(r, m) = (1, 1)$

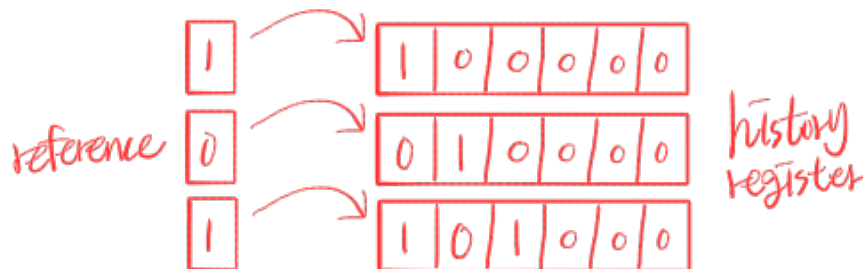
LRU Approximation Algorithms

- NUR algorithm



LRU Approximation Algorithms

- Additional reference-bits algorithm (NUR) history
 - LRU approximation
 - History register for each page
 - Recording the reference bits at regular intervals
 - Timer interrupts at regular intervals
 - OS shifts the reference bit for each page into the high-order bit of its history register, shifting the other bits right by one bit and discarding the low-order bit
 - Replacement based on the value of history register
 - Interpret the value of the history register as unsigned integers
 - Choose the page with smallest value as a victim



LRU Approximation Algorithms

- **Additional reference-bits algorithm**
- Example
 - Value of the 8-bit history registers
 - 00000000
 - No reference during 8 time periods
 - 11111111
 - Referenced at least once in each period
 - Page a with 01100100 and page b with 00110111
 - Page a has been used more recently than page b

victim = b

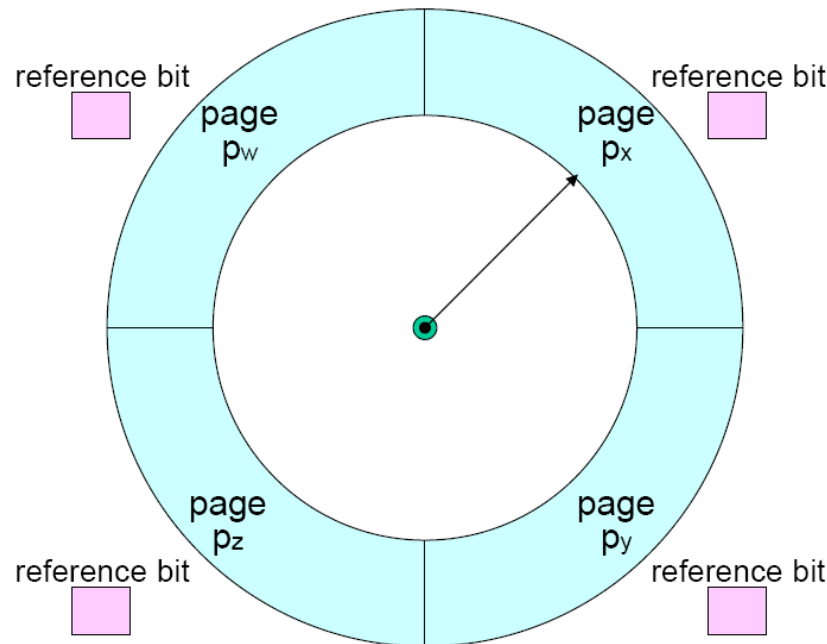
LRU Approximation Algorithms

- **Clock algorithm (Second-chance algorithm)** *Very famous*

- Used for IBM VM/370 OS
- Uses reference bit

NUR과 달리 • No periodical reset for reference bits

- Choose the page to be replaced using pointer that circulates the list of pages (page frames) in memory



LRU Approximation Algorithms

- **Clock algorithm**

- Scheme

- Choose the page to be replaced with the pointer moving clockwise

- Mechanism



- ① Check the reference bit r of the page that the pointer points

- ② If $r == 0$, select the page as a victim

- ③ If $r == 1$, reset the reference bit to 0 and advances the pointer goto step-①

reference bit → 0이면 접근 기록은 access되지 않았다는 의미 → victim

- Characteristics

- Earlier memory load time → higher probability of displacement

- Similar to FIFO algorithm

- Page replacement based on the reference bit

- Similar to LRU (or NUR) algorithm

LRU Approximation Algorithms

• Clock algorithm: Example

- Assumptions
 - 4 page frames are allocated to the process
 - Initially, it has pages a, b, c, d in memory
 - Reference bits of the 4 page frames are all 1

$\omega = c a d b e b a b c d$

AB 등판해 1호 관련

Memory state change (Clock)

Time	0	1	2	3	4	5	6	7	8	9	10
Ref. string		c	a	d	b	e	b	a	b	c	d
Memory state	frame 0	a/1	→a/1	→a/1	→a/1	e/1	e/1	e/1	e/1	→e/1	d/1
	frame 1	b/1	b/1	b/1	b/1	→b/0	→b/1	b/0	b/1	→b/1	→b/0
	frame 2	c/1	c/1	c/1	c/1	c/0	c/0	a/1	a/1	a/1	a/0
	frame 3	d/1	d/1	d/1	d/1	d/0	d/0	→d/0	→d/0	→c/1	c/0
Page fault						F		F		F	F
Pclock						e		a		c	d
Qclock						a		c		d	e

- Pclock : Pages loaded into memory
- Qclock : Pages displaced from memory

이리 해서 pointer가 끝난 이동
no period

LRU Approximation Algorithms

- **Enhanced clock algorithm**

- Similar to clock algorithm
- Considers **update (dirty, modified) bit** as well as reference bit

- Scheme

- Choose the page to be replaced with the pointer moving clockwise
- Mechanism

① Check (r, m) of the page that the pointer points

② $(0, 0)$: select the page as a victim, advances the pointer

③ $(0, 1)$: set (r, m) to $(0, 0)$, put the page on the cleaning list, goto step-⑥

dirty → clean

→ write → storage

④ $(1, 0)$: set (r, m) to $(0, 0)$, goto step-⑥

⑤ $(1, 1)$: set (r, m) to $(0, 1)$, goto step-⑥

⑥ Advances the pointer, goto step-①

$(1, 1) \rightarrow (0, 1)$

$(1, 0) \rightarrow (0, 0) \Rightarrow \text{victim}$

LRU Approximation Algorithms

Enhanced clock algorithm: Example

Assumptions

- 4 page frames are allocated to the process
- Initially, it has pages a, b, c, d in memory
- All reference bits are 1, all update bits are 0

$\omega = c a^w d b^w e b a^w b c d$

read write operation
Hit → ref, modify

Memory state change (Enhanced clock)

Time		0	1	2	3	4	5	6	7	8	9	10
Ref. string		<i>access, clean</i> c	<i>Hit read</i> c	<i>Hit</i> a ^w	<i>Hit</i> d	<i>Hit write</i> b ^w	e	<i>Hit</i> b	<i>Hit</i> a ^w	<i>Hit</i> b	<i>write</i> c	d
Memory state	frame 0	→ a/10	→ a/10	→ a/10	→ a/11	→ a/11	a/00	a/00	a/11	a/11	→ a/11	a/00
	frame 1	b/10	b/10	b/10	b/10	b/10	b/00	b/10	b/10	b/10	b/10	d/10
	frame 2	c/10	c/10	c/10	c/10	c/10	e/10	e/10	e/10	e/10	e/10	→ e/00
	frame 3	d/10	d/10	d/10	d/10	d/10	d/00	→ d/00	→ d/00	→ d/00	c/10	c/00
Page fault						<i>P2nd-chance</i> F	F				F	F
P2nd-chance						e	e				c	d
Q2nd-chance						c	c				d	b

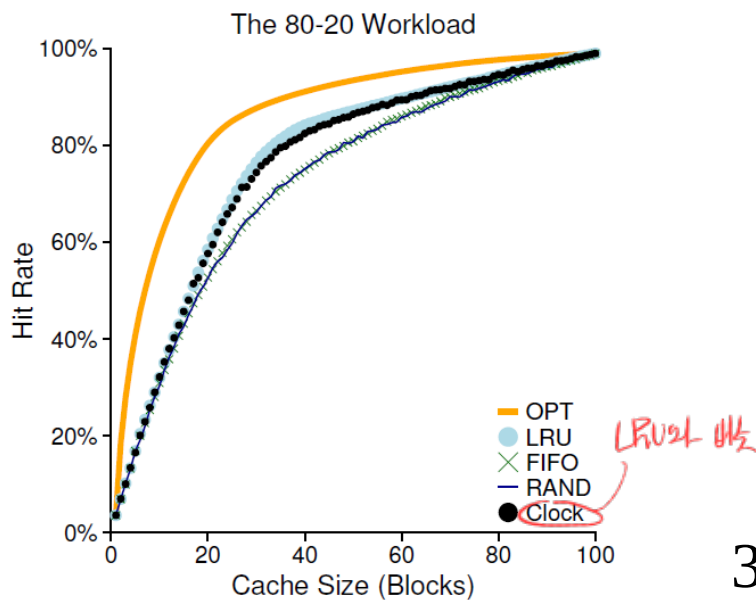
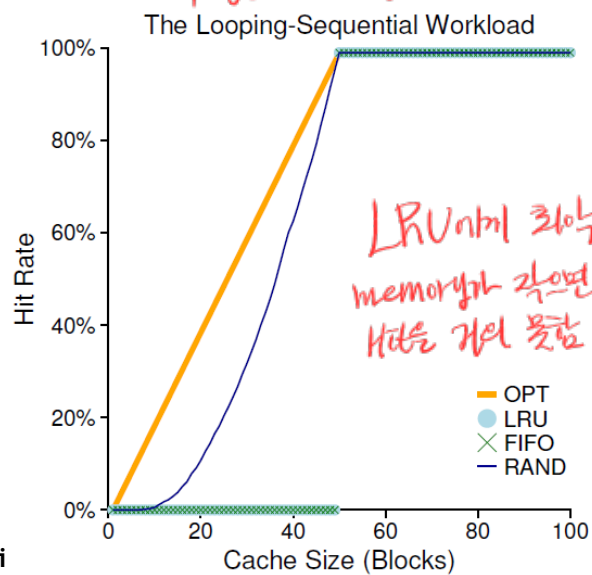
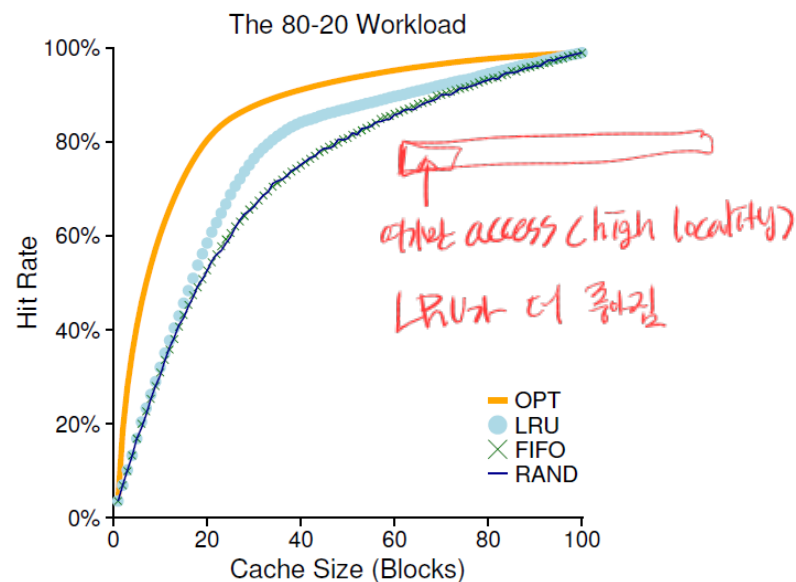
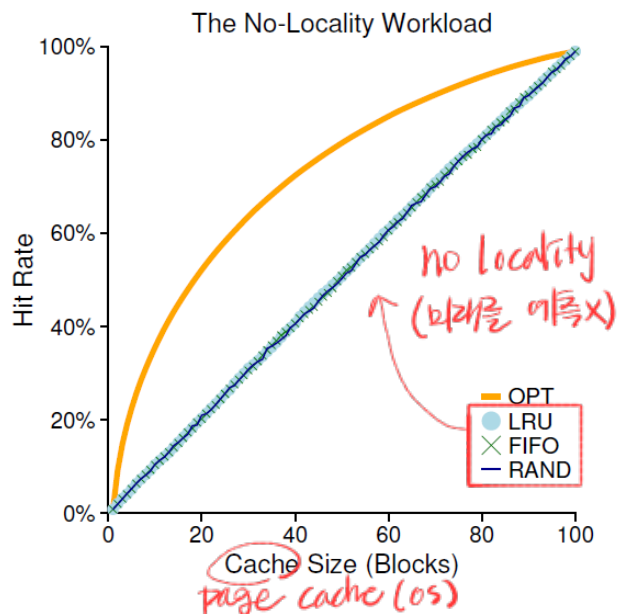
- Superscript W : page update
- Underline : the page is on the cleaning list
- P2nd-chance : pages loaded into memory
- Q2nd-chance : pages displaced from memory

Counting Algorithms

- Keep a counter of the number of references that have been made to each page
– Not common *access 횟수 기록*
- **Least Frequently Used (LFU) Algorithm**
– replaces page with smallest count *가장 작게 access되는 page*
- **Most Frequently Used (MFU) Algorithm**
– based on the argument that the page with the smallest count was probably just brought in and has yet to be used
young page

Workload Examples

Hit rates

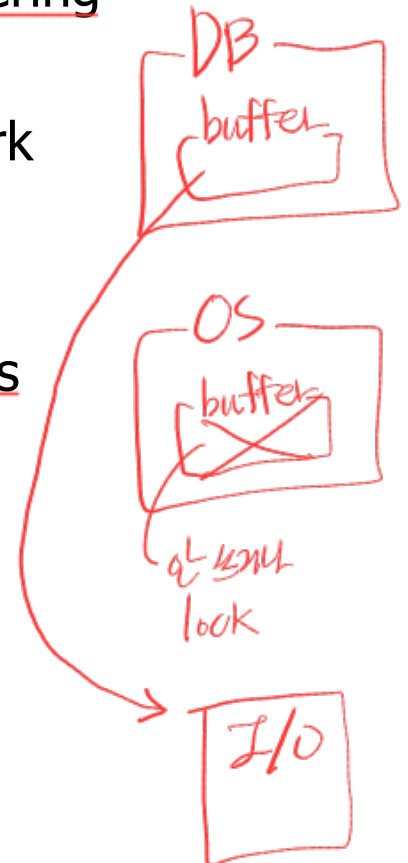


Applications and Page Replacement

- All of these algorithms have OS guessing about future page access
- Some applications have better knowledge – i.e. databases
- Memory intensive applications can cause double buffering
 - OS keeps copy of page in memory as I/O buffer
 - Application keeps page in memory for its own work
- Bypasses buffering, locking, etc
- **O_DIRECT** mode
 - File I/O is done directly to/from user-space buffers
 - `fd=open(pathname, O_RDWR | O_DIRECT )`

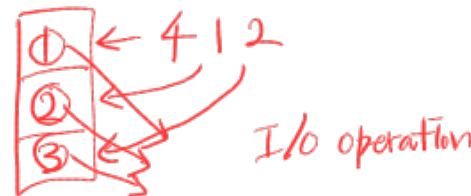
double
buffering
problem

predict future access



Thrashing

- If a process does not have “enough” pages, the page-fault rate is very high
 - Page fault to get page → Replace existing frame → But quickly need replaced frame back
 - This leads to:
 - Low CPU utilization
 - Operating system thinking that it needs to increase the degree of multiprogramming
 - Another process added to the system
- **Thrashing**
 - a process is busy swapping pages in and out



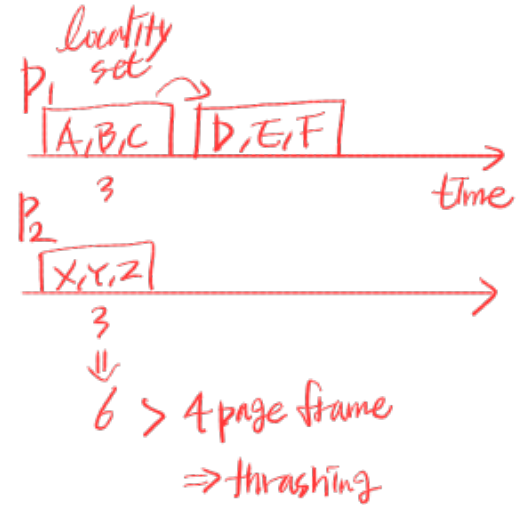
Demand Paging and Thrashing

- Why does demand paging work?

- Locality model

- Spatial locality and Temporal locality

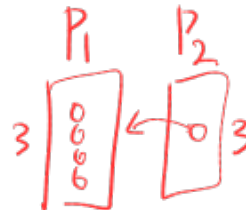
- Process migrates from one locality to another
 - Localities may overlap



- Why does thrashing occur?

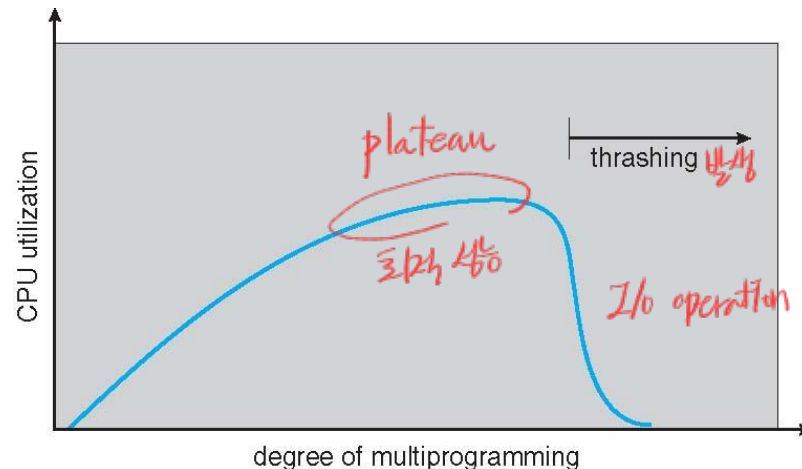
- Σ size of locality $>$ total memory size
 - Limit effects by using local or priority page replacement
 - If one process starts thrashing, it cannot steal frames from another process
 - However, the average service time for a page fault will increase

다른 process의 frame을 빼앗지 못해 성능이 저하되기 않는다



Admission control

- **Underloaded** *system이 느리고 있는 상태* *throughput*
 - System resource waste, performance degradation
- **Overloaded** *너무 많은 process 실행*
 - System resource contention, performance degradation
 - Thrashing (excessive paging activity) *메모리 부족* *page fault* *disk 접근 (I/O operation 증가)*
- Should maintain multiprogramming degree at the appropriate range
 - Plateau ranges are located at different positions for different systems *각각에게 실행해야 할*
- A harsher approach: out-of-memory killer
 - when memory is oversubscribed, a memory-intensive process is killed *메모리를 많이 쓰는 process를 kill*



Be Lazy: Demand Zeroing

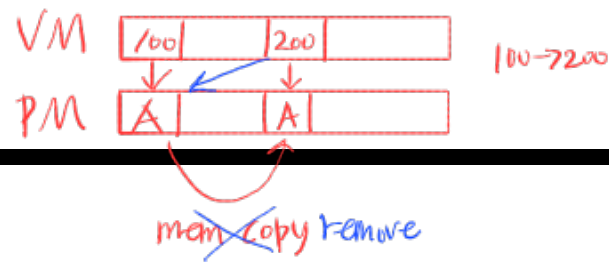
- To add a page to your address space (heap)
 - OS adds a page to your heap by finding a page in physical memory,
 - zeroing it (required for security)
 - and then mapping it into your address space (i.e., setting up the page table).
 - High cost particularly if the page does not get used
- Demand zeroing
 - OS puts an entry in the page table that marks the page inaccessible
 - If the process then reads or writes the page, a trap takes place.
 - When handling the trap, the OS notices that this is actually a demand-zero page.
 - at this point, the OS then does the needed work of finding a physical page, zeroing it, and mapping it into the process's address space.
 - If the process never accesses the page, all of this work is avoided

physical memory를 0으로 할당할 뒤
→ address space에 mapping
→ 실제로 안 쓰일 수 있음

inaccessible
↓
ready, write
↓

위의 작업 3개

Be Lazy: Copy-on-Write



- **Copy-on-Write**

- When OS needs to copy a page from one address space to another, instead of copying it, it can map it into the target address space and mark it read-only in both address spaces. → fast copy
- For read operation, no further action is taken *LOW FA*
- For write operation, a trap occurs *다음에는 read-only, 페이지 공유함*
 - OS (lazily) allocate a new page, fill it with the data, and map this new page into the address space of the faulting process.

- **fork()** system call

쓰레드가 발생 → trap → 새로운 페이지를 복사 → write

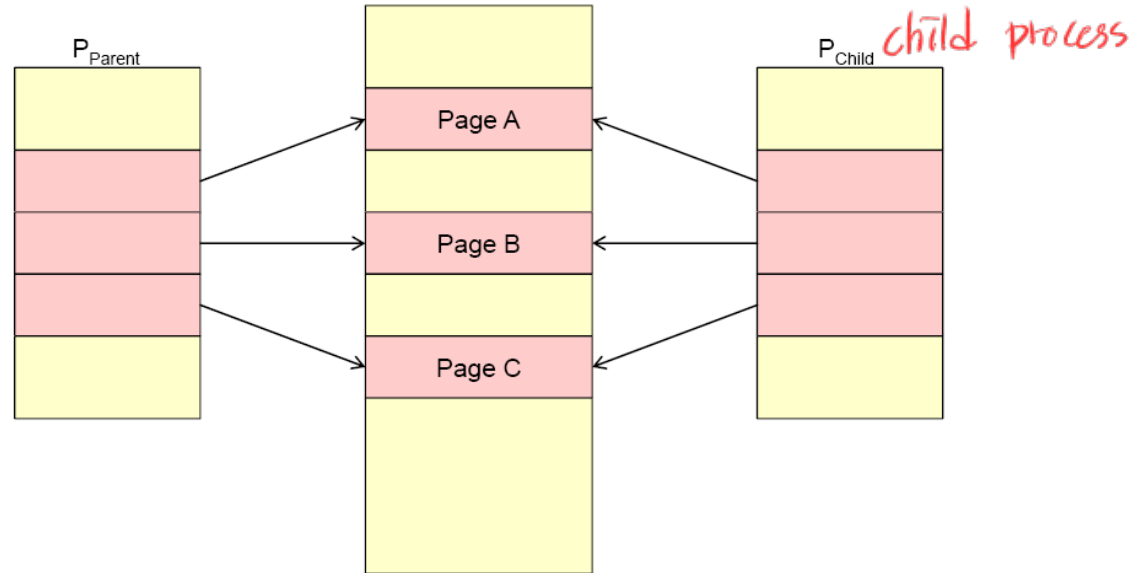
- Creates a child process as a duplicate of its parent
- Should create a copy of the parent's address space for the child, duplicating the pages of the parent

- With **copy-on-write** scheme

- Allows the parent and the child processes initially share the pages (marked as copy-on-write)
- When any process writes to a shared page, a copy of the shared page is created

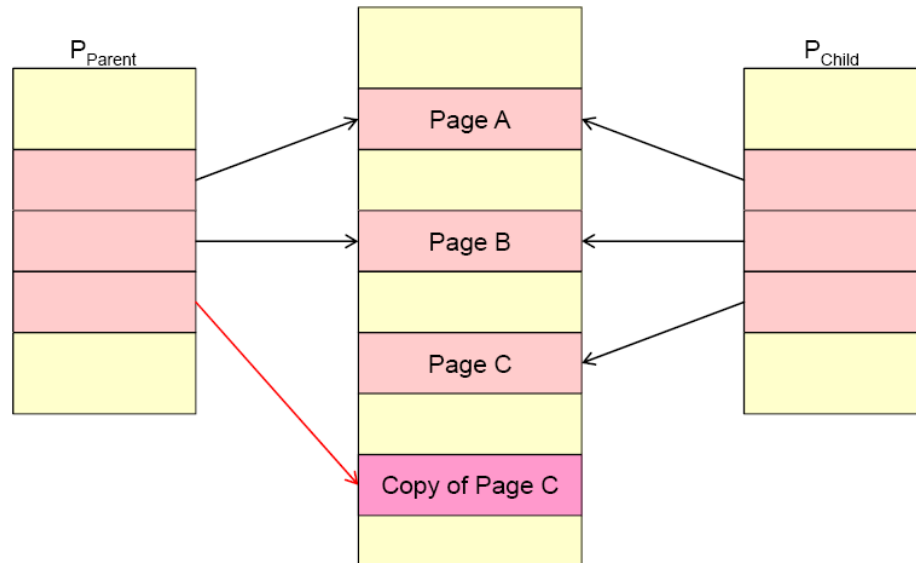
Be Lazy: Copy-on-Write

After **fork()** system call



After Parent modifies Page-C

Write한 쪽에 새 page를
mapping



Be Lazy: Copy-on-Write

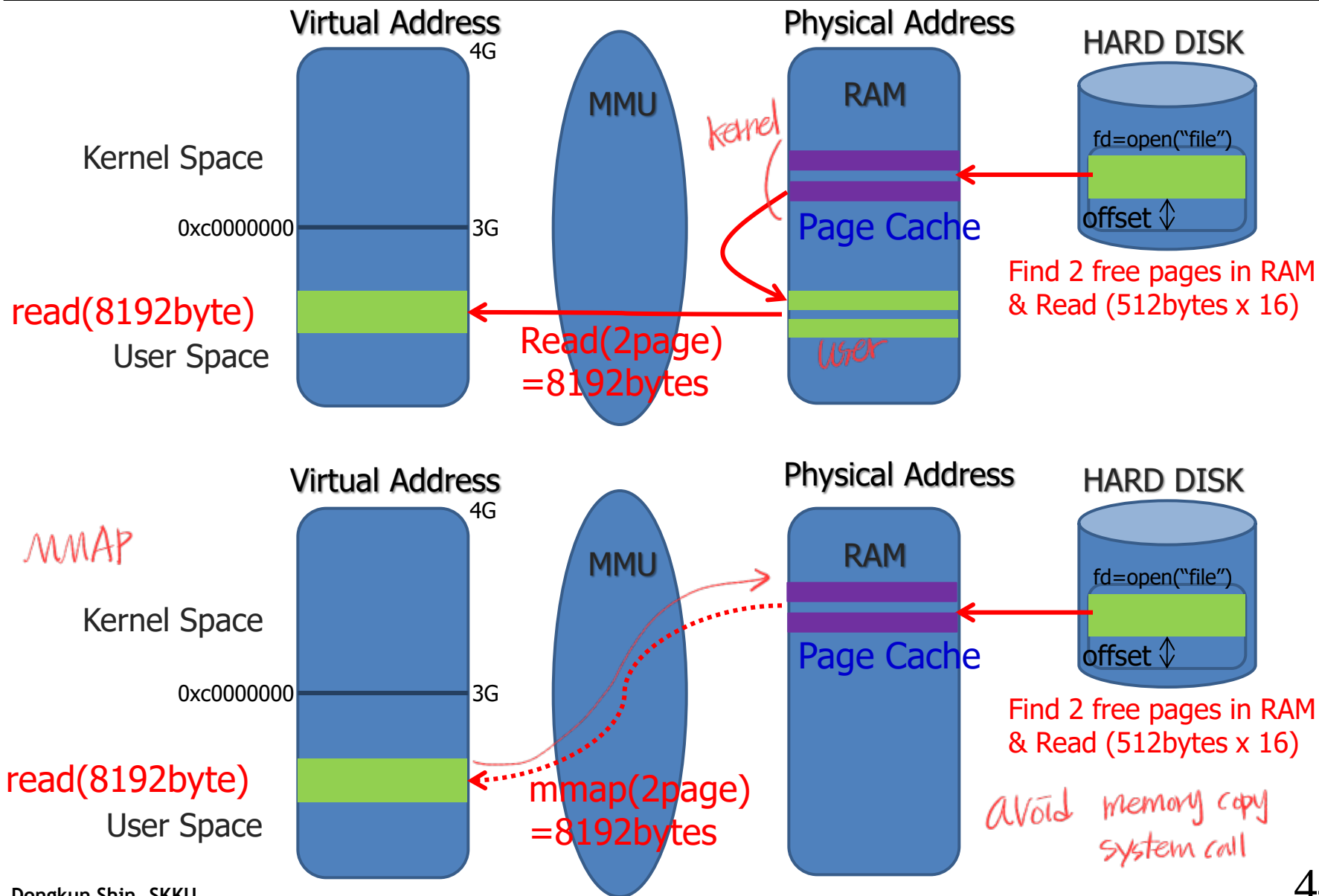
- **Notes on copy-on-write**

- Only the copy-on-write pages that are modified by either process are copied
 - All unmodified pages are shared by the parent and child processes
- Only the pages that can be modified can be marked as copy-on-write
- Operating systems that use copy-on-write scheme
 - Linux, Solaris, Windows XP

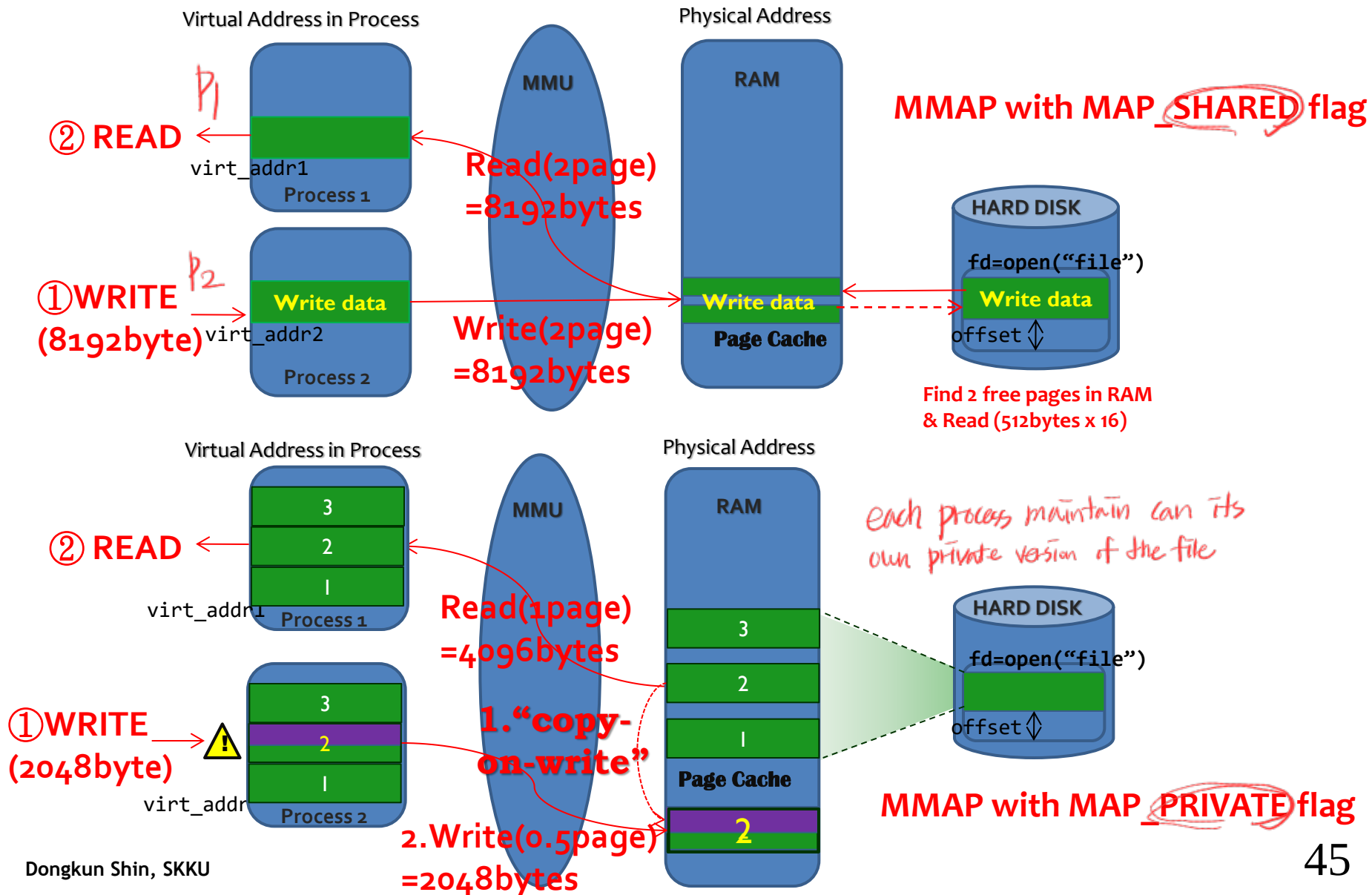
Memory-Mapped Files

- **Memory-mapped file I/O** allows **file I/O** to be treated as routine memory access by **mapping** a disk block to a page in memory
disk block을 mapping해서 memory access처럼 처리
- **A memory-mapped file is read using demand paging**
 - Accesses to parts of a memory-mapped file that have not yet been brought into memory trigger **page faults**
 - A page-sized portion of the file is read from the file system into a physical page
 - Subsequent reads/writes to/from the file are treated as ordinary memory accesses
- **Simplifies and speeds file access by driving file I/O through memory rather than read() and write() system calls**
- **Also allows several processes to map the same file allowing the pages in memory to be shared** *여러 process가 page 공유*
- **But when does written data make it to disk?**
 - Periodically and / or at file **close()** time (**munmap**)
 - For example, when the pager scans for dirty pages
 - **msync()** *즉시 write할*
*주기적으로 page scan, file close(munmap) → write할**read, write 없이
간접 memory access
더 빠를 것*

MMAP



MMAP on Shared File



Other Considerations -- Prepaging

- **Prepaging (Prefetch)**
 - To reduce the large number of page faults that occurs at process startup
*process 실행 도중에 page fault가 빈번할
process가 실행되기 전 사용될 것 같은 pages를 미리 불러들*
 - Prepage all or some of the pages a process will need, before they are referenced
predict
 - But if prepaged pages are unused, I/O and memory was wasted
 - Assume s pages are prepaged and a of the pages is used
 - Is cost of $s * a$ save pages faults > or < than the cost of prepaging
(미리 불러서 피한 page faults)
 - $s * (1 - a)$ unnecessary pages?
낭비한 자원
 - a near zero $\Rightarrow$ prepaging loses
 a 가 0에 가까울 수록 prepaging이 불리함

Other Issues – Page Size

- **Sometimes OS designers have a choice**
 - Especially if running on custom-built CPU
- **Page size selection must take into consideration:**

- Fragmentation
- Page table size
- Resolution
- I/O overhead
- Number of page faults
- Locality
- TLB size and effectiveness

Smaller page size

- + Smaller internal fragmentation
- + Match program locality more accurately
 - o Reduction in total I/O
 - o Better utilization of memory
(Less total allocation of memory)
- Larger page table size
- Increase in I/O time
- Increase in the number of page faults

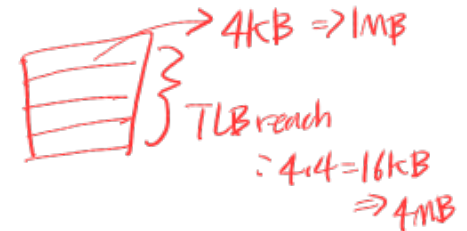
- **Always power of 2, usually in the range 2^7 (128 bytes) to 2^{22} (4MB)**
- **On average, growing over time**

popular size : 4KB

Other Issues – TLB Reach (Coverage)

- **TLB Reach**

- The amount of memory accessible from the TLB
- $\text{TLB Reach} = (\text{TLB Size}) \times (\text{Page Size})$



- **Ideally, the working set of each process is stored in the TLB**

- Otherwise there is a high degree of TLB misses

- **Increase the number of entries in TLB**

- Expensive

- **Increase the Page Size**

- lead to an increase in fragmentation as not all applications require a large page size

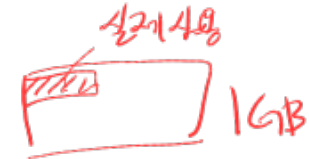
- **Provide Multiple Page Sizes**

- Allows applications that require larger page sizes the opportunity to use them without an increase in fragmentation
- Requires OS support
 - One of the fields in TLB entry must indicate the size of the page (frame) corresponding to the TLB entry



Large Page Support

- Intel x86 allows for multiple page sizes *4KB, 2MB, 1GB*
- Linux supports huge pages *x64: 4, 8, 16, 256, ...*
- Huge page *→ 2MB, 1GB*
 - reduces the number of mappings in the page table
 - the larger the pages, the fewer the mappings.
 - Better TLB behavior and performance gains.
 - Shorter TLB-miss path (served more quickly)
 - Applications explicitly request memory allocations with large pages (either through mmap() or shmget())
 - Internal fragmentation
 - *불러* Amplifying the amount of I/O at swapping
- **Transparent huge page**
 - OS automatically looks for opportunities to allocate huge pages without requiring application modification



reduce mapping entries

Other Issues – Program Structure

- **Program restructuring**

- System performance can be improved if the user (or compiler/linker/loader) has an awareness of the paged nature of memory or underlying demand paging system
- Program restructuring by user or compiler/linker/loader

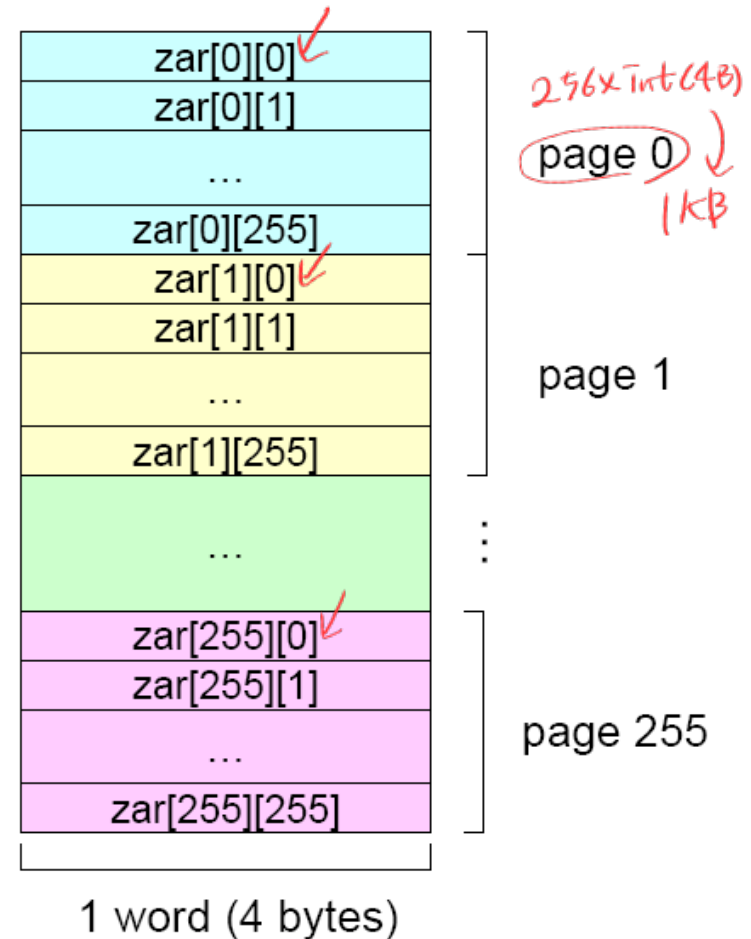
Other Issues – Program Structure

- **Example**

- Assumptions
 - Paging system of 1KB page size
 - sizeof(int): 4 Bytes

```
// Program-1
int main()
{
    int zar[256][256];
    int i, j;

    for(j = 0; j < 256; j++)
        for(i = 0; i < 256; i++)
            zar[i][j] = 0;
    return 0;
}
```



For each execution of loop instance, pages 0~255 are referenced sequentially and this process is repeated 255 times

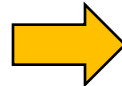
For execution with no page faults, The process should have 256 page frames

- Not practical

Other Issues – Program Structure

```
// Program-1
int main()
{
    int zar[256][256];
    int i, j;

    for(j = 0; j < 256; j++)
        for(i = 0; i < 256; i++)
            zar[i][j] = 0;
    return 0;
}
```

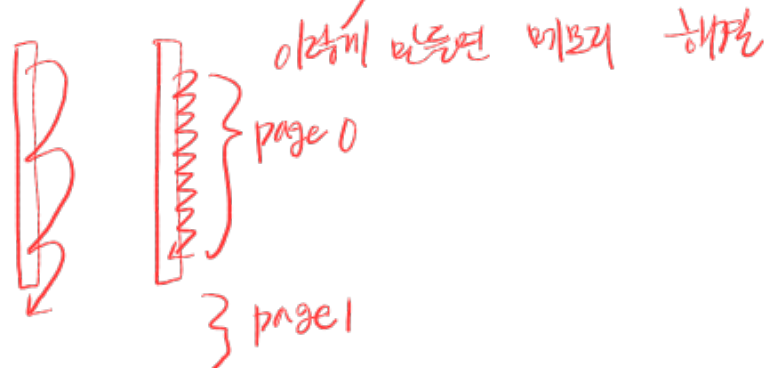


loop
inter
change

```
// Program-2
int main()
{
    int zar[256][256];
    int i, j;

    for(i = 0; i < 256; i++)
        for(j = 0; j < 256; j++)
            zar[i][j] = 0;
    return 0;
}
```

higher
spatial
locality



Other Issues – Program Structure

- Notes

- User

- Careful selection of data structures and programming structures can increase locality (hence lower page-fault rate)

- Stack: has good locality

- Hash table: produces bad locality

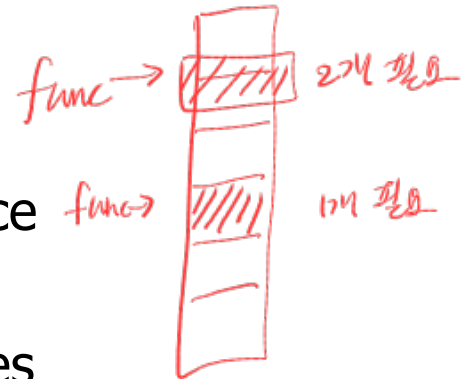
- Pointer: diminishes locality

- OOPs tend to have a poor locality of reference

- Linker/Loader

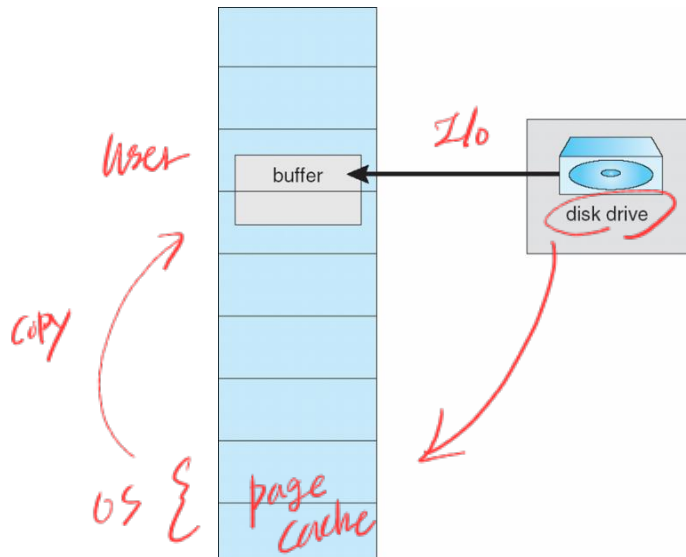
- Avoid placing routines across page boundaries

- Let routines that call each other many times be packed into the same page



Other Issues – I/O interlock

- **I/O Interlock** – Pages must sometimes be locked into memory
 - Consider I/O - Pages that are used for copying a file from a device must be locked from being selected for eviction by a page replacement algorithm (lock bit)
 - Another solution: never to execute I/O to user memory
- **Pinning** of pages to lock into memory



Homework

- Homework (**Measurement**) in Chap 21 (**vmstat**)
- Homework (**Simulation**) in Chap 22 (paging-policy.py)

MMAP example

```
#include <sys/mman.h>
#include <fcntl.h>
#include "tlpi_hdr.h"

#define MEM_SIZE 10
int main(int argc, char *argv[])
{
    char *addr;
    int fd;

    fd = open(argv[1], O_RDWR);
    if (fd == -1) errExit("open");
    addr = mmap(NULL, MEM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (addr == MAP_FAILED) errExit("mmap");
    if (close(fd) == -1) errExit("close"); /* No longer need 'fd' */

    printf("Current string=.*s\n", MEM_SIZE, addr);
    if (argc > 2) { /* Update contents of region */
        if (strlen(argv[2]) >= MEM_SIZE) cmdLineErr("'new-value' too large\n");
        memset(addr, 0, MEM_SIZE); /* Zero out region */
        strncpy(addr, argv[2], MEM_SIZE - 1);
        if (msync(addr, MEM_SIZE, MS_SYNC) == -1) errExit("msync");
        printf("Copied \"%s\" to shared memory\n", argv[2]);
    }
    exit(EXIT_SUCCESS);
}
```

\$./t_mmap s.txt hello

Current string= null

Copied "hello" to shared memory

\$./t_mmap s.txt goodbye

Current string=hello

Copied "goodbye" to shared memory

named pipe

파일기반 공유 메모리

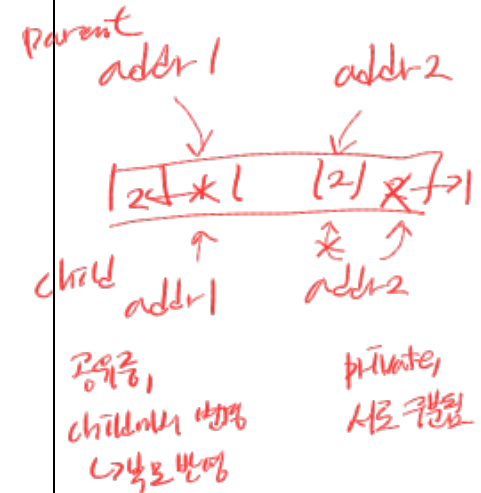
↳ 디스크에 저장

MMAP example

```
#include <sys/wait.h>
#include <sys/mman.h>
#include <fcntl.h>
int main(int argc, char *argv[])
{
    int *addr1, *addr2;    /* Pointer to shared memory region */
    /* Parent creates mapped region prior to calling fork() */
    addr1 = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
        MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    addr2 = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
        MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (addr1 == MAP_FAILED || addr2 == MAP_FAILED) errExit("mmap");
    *addr1 = 1; *addr2 = 2; /* Initialize integer in mapped region */
    printf("Initially, v1 = %d, v2 = %d \n", *addr1, *addr2);

    switch (fork()) { /* Parent and child share mapping */
    case -1:
        fprintf(stderr, "fork failed\n");
        exit(1);
    case 0: /* Child: increment shared integer and exit */
        (*addr1)++; (*addr2)--;
        printf("Child updated, v1 = %d, v2 = %d \n", *addr1, *addr2);
        munmap(addr1, sizeof(int)); munmap(addr2, sizeof(int));
        exit(1);
    default: /* Parent: wait for child to terminate */
        wait(NULL);
        printf("Parent read, v1 = %d, v2 = %d \n", *addr1, *addr2);
        munmap(addr1, sizeof(int)); munmap(addr2, sizeof(int));
        exit(1);
    }
}
```

```
$ ./anon_mmap
Initially, v1 = 1, v2 = 2
Child updated, v1 = 2, v2 = 1
Parent read, v1 = 2, v2 = 2
```



→ mapping된 memory 공유 (Shared memory mapping)
 (Child process only uses its own mapping)
 'child'

MMAP

- `pmap` - report memory map of a process

virtual address	size	protection	
00000000000400000	372K	r-x--	tcsh
000000000019d5000	1780K	rw---	[anon] heap
00007f4e7cf06000	1792K	r-x--	libc-2.23.so
00007f4e7d2d0000	36K	r-x--	libcrypt-2.23.so
00007f4e7d508000	148K	r-x--	libtinfo.so.5.9
00007f4e7d731000	152K	r-x--	ld-2.23.so
00007f4e7d932000	16K	rw---	[stack]

mmapped

Linux Virtual Memory System

- **The kernel portion is the same across processes** 모든 process가 같은 kernel을 공유
- **Kernel logical address**
 - Normal virtual address space of the kernel
 - Allocated by kmalloc()
 - Most kernel data structures
 - page tables, per-process kernel stacks, ...
 - cannot be swapped to disk.
 - Direct mapping (logical addr → physical addr)
 - 0xC0000000 → 0x00000000, 0xC0000FFF → 0x00000FFF, ...
 - Simple translation (w/o support of PMT)
 - Logically contiguous → physically contiguous (useful for DMA)
- **Kernel virtual address**
 - Allocated by vmalloc()
 - Usually not contiguous; each kernel virtual page may map to non-contiguous physical pages (and is thus not suitable for DMA).
 - Easier to allocate a large buffer

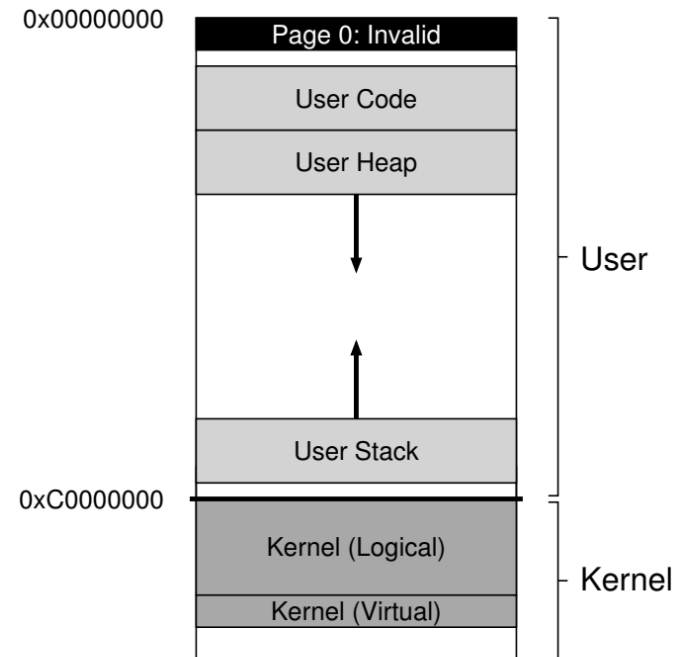


Figure 23.2: The Linux Address Space

E1/2X
TLBX

Linux Page Cache

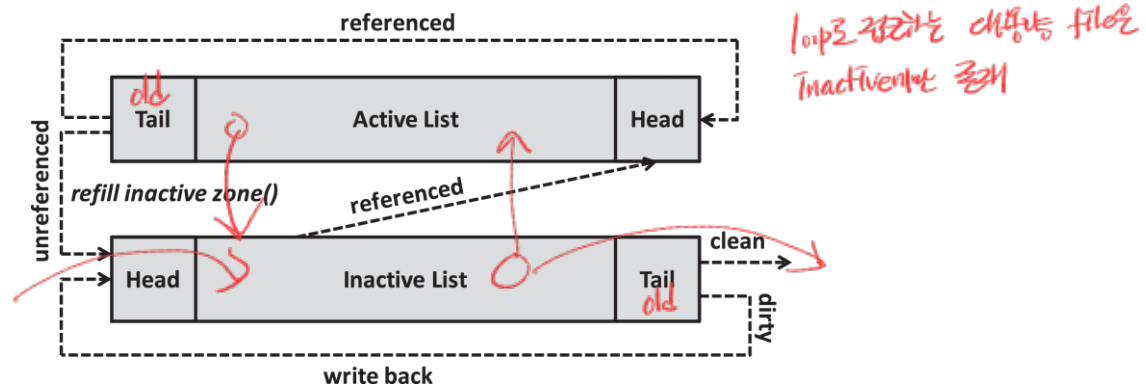
- **Unified page cache keeping pages in memory from three primary sources**
 - Memory-mapped files (mmap())
 - file data and metadata from devices (read() and write())
 - heap and stack pages (anonymous memory)
 - no named file underneath of it, but rather swap space.
- **Page cache hash table for quick lookup**
- **Dirty page flush**
 - periodically written to the backing store to
 - a specific file for file data or swap space
 - by background threads (called pdflush)
 - after a certain time period (dirty_expire_centisecs) or
 - if too many pages are dirty (dirty_background_ratio)

양성사는여 귀하고

dirty 비율이 높아지면

Diagram illustrating the relationship between a page cache and a file. The page cache contains a page with a pointer to a file. The file contains a pointer to the page cache.

- 전체 내뱉고 나서 reference하는 법이 귀찮음 (순차적인 접근)



Quiz1

1. Page faults are handled entirely by hardware with no OS involvement. (X)
2. The page table is updated after a page fault is handled. (O)
3. LRU (Least Recently Used) page replacement algorithm replaces the page that has not been used for the longest time. (O)
4. Increasing the number of page frames always reduces the number of page faults. (X)
5. When free memory rises above the high watermark, the OS halts page replacement activity. (O)
6. Low watermark refers to the threshold below which the system begins paging out memory. (O)

Quiz2

1. In the Clock LRU algorithm, a reference bit is managed for each page frame. (O)
2. Clock LRU always removes the page with the longest time since last use. (X)
3. If a page's reference bit is set to 1, the clock algorithm gives it a second chance and clears the bit. (O)
4. The Clock LRU algorithm is simpler than the Optimal algorithm to implement. (O)
5. ~~O_DIRECT~~ ensures data consistency by keeping a copy of the data in both the page cache and the user buffer. (X)
6. Page thrashing occurs when the system spends more time handling page faults than executing processes. (O)
7. Demand zeroing is used to delay physical page allocation until needed. (O)
8. The fork() system call uses Copy-On-Write to avoid immediate duplication of memory. (O)
9. Page protection bits in PTE are essential for implementing Copy-On-Write. (O)
10. The changes made to an mmap-ed region are always ~~immediately~~ written to disk. (X)

asynch

Quiz2

- 11. MAP_ANONYMOUS flag is used to create a mapping not backed by a file. (O)
- 12. MAP_PRIVATE flag ensures changes to memory are ~~visible~~ to other processes. (X)
- 13. It is possible to use multiple page sizes simultaneously. (O)
- 14. Larger page sizes can reduce the size of page tables. (O)
- 15. Smaller pages lead to ~~higher~~ internal fragmentation. (X)
- 16. Larger pages can increase I/O efficiency during page faults. (O) *No time to*
- 17. Increasing the page size extends the TLB reach and can eventually improve the TLB hit rate. (O)
- 18. Linux supports both explicit huge pages and transparent huge pages. (O)
- 19. The mlock() system call can be used by user-space processes to pin pages in memory. (O)
- 20. Loop interchange can be used to improve ~~temporal~~ *spatial* locality. (X)